

22. live forecast 取得ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/weather_fetch_node.py
種別	Python
区分	runtime node
役割	chase GPS または fallback 座標から Open-Meteo forecast を取得し、planner が読む CSV を更新する。
起動文脈	live 系の forecast 更新入口。

このファイルを読む理由

chase GPS または fallback 座標から Open-Meteo forecast を取得し、planner が読む CSV を更新する。

- raw forecast CSV を定期更新する。
- planner 自体は topic ではなく forecast CSV を読む。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/live_launch.py

次に読むと理解が進むファイル

- mpc_solarcar/weather_utils.py
- mpc_solarcar/wind_correction_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py
- mpc_solarcar/weather_utils.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .weather_utils import fetch_openmeteo_forecast, write_forecast_csv

class WeatherFetchNode(Node):
    def __init__(self) -> None:
        super().__init__("weather_fetch_node")
        self.declare_parameter("profile_yaml", "")
        self.declare_parameter("output_csv", "")
        self.declare_parameter("raw_forecast_csv", "")
        self.declare_parameter("gps_topic", "/chase/gps")
        self.declare_parameter("fetch_period_sec", 3600.0)
        self.declare_parameter("forecast_days", 3)
        self.declare_parameter("step_minutes", 10)
        self.declare_parameter("timezone_name", "Australia/Darwin")
        self.declare_parameter("fallback_latitude", -12.4634)
        self.declare_parameter("fallback_longitude", 130.8456)
        self.declare_parameter("tcell_gain", 0.03)

        profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
        if profile_yaml:
            profile_path, cfg = load_profile(profile_yaml)
            runtime_cfg = get_section(cfg, "runtime")
```

主要構造の読み取り

主要クラスは WeatherFetchNode。主要関数は main。ROS パラメータ宣言は 11 件。ROS I/O は publisher 1 件、subscription 1 件。

主要クラス・関数

行	種別	名前	補足
15	class	WeatherFetchNode	bases: Node
16	function	__init__	args: self
104	function	_on_gps	args: self, msg
111	function	_fetch_once	args: self
138	function	main	

ファイルを上から読んだときの定義順

行	内容
15	クラス WeatherFetchNode を定義する。
138	関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L85, L86。
5	<code>importrcipy</code>	ROS 2 Python ノードとして起動・ <code>spin</code> するため。このファイル内での主な使用位置は L39, L40, L44, L48, L52, L56, L60, L69, ...。
6	<code>fromrcipy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L15。
8	<code>fromsensor_msgs. msgimportNavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L99, L104。
9	<code>fromstd_msgs. msgimportString</code>	Float32 や String など軽量メッセージで <code>planner / vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L98, L130, L134。
11	<code>from.solar_ profileimportget_path,get_ section,load_profile</code>	profile YAML 読込と検証から <code>get_path</code> , <code>get_section</code> , <code>load_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L32, L33, L34, L35, L39。
12	<code>from.weather_ utilsimportfetch_openmeteo_ forecast,write_forecast_csv</code>	<code>weather_utils.py</code> から <code>fetch_openmeteo_forecast</code> , <code>write_forecast_csv</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/weather_utils.py</code> 。このファイル内での主な使用位置は L115, L123, L128。

関数・クラスを上から順に解説

L15 クラス WeatherFetchNode

- 定義: `WeatherFetchNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Parameter`, `Path`, `String`, `__init__`, `_fetch_once`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `error`, `expanduser`, `fetch_openmeteo_forecast`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `_on_gps` を定義する。
3. 関数 `_fetch_once` を定義する。

L138 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `WeatherFetchNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. `rcipy.init(...)` を実行する。
2. `node` に `WeatherFetchNode()` の結果を代入する。
3. `rcipy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rcipy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
18	<code>profile_yaml</code>	
19	<code>output_csv</code>	
20	<code>raw_forecast_csv</code>	
21	<code>gps_topic</code>	<code>/chase/gps</code>
22	<code>fetch_period_sec</code>	3600.0
23	<code>forecast_days</code>	3
24	<code>step_minutes</code>	10
25	<code>timezone_name</code>	Australia/Darwin
26	<code>fallback_latitude</code>	-12.4634
27	<code>fallback_longitude</code>	130.8456
28	<code>tcell_gain</code>	0.03

ROS topic I/O

行	種別	topic	callback / 補足
98	publisher	<code>/weather/fetch_status</code>	publisher
99	subscription	<code>self.gps_topic</code>	<code>self._on_gps</code>

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/weather_fetch_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。